

Apache camel

What is Apache Camel?

Apache Camel is an open source integration framework that aims to make integrating systems easier. At the core of the Camel framework is a **routing engine**, or more precisely a routing-engine builder. It allows you to define your own routing rules, decide from which sources to accept messages, and determine how to process and send those messages to other destinations.

Camel has two main ways of defining routing rules:

- Java-based domain-specific language (DSL)
- Spring XML configuration format.

Java-based domain-specific language (DSL) : -

A Java-based domain-specific language (DSL) is a programming language designed specifically to solve problems within a particular domain or application area. DSLs provide a way to express complex business rules or requirements in a more intuitive and natural way than is possible with general-purpose programming languages like Java.

Java-based DSLs are written in Java and typically use a fluent interface or a builder pattern to provide a high-level, domain-specific syntax for expressing complex computations or workflows. The DSL can be embedded within a Java application or library, or it can be a standalone language that is interpreted or compiled into Java bytecode.

Spring XML configuration format:-

Spring XML configuration format is an XML-based format used to configure Spring Framework applications. It is the original way to configure Spring applications and is still widely used today, although it has been largely replaced by Java configuration and annotation-based configuration in recent versions of Spring.

The Spring XML configuration file typically has a root element `<beans>` which contains one or more nested bean definitions, each represented by a `<bean>` element. The `<bean>` element specifies the class or interface of the bean, along with any constructor arguments or property values that should be injected into the bean.

Apache Camel use cases:-

- ➔ **Enterprise application integration (EAI):** - Camel can be used to integrate different applications and systems across an enterprise, allowing data and messages to be exchanged between different systems in a seamless and consistent way.
- ➔ **Message-driven architecture (MDA):** - Camel can be used to implement a message-driven architecture, where applications and services communicate through messages rather than direct method invocations.
- ➔ **Microservices integration :-** Camel can be used to integrate microservices by providing a flexible and lightweight message-based communication layer between different services.
- ➔ **Data transformation:** - Camel can be used to transform data from one format to another, such as transforming data from a database into a message format that can be sent to a messaging system.

- **ETL (extract, transform, load):** - Camel can be used to implement ETL workflows, where data is extracted from a source system, transformed into a different format, and loaded into a target system.
- **API integration:** - Camel can be used to integrate with APIs, providing a flexible and customizable way to manage and interact with APIs.

Overall, Apache Camel is a powerful and flexible integration framework that can be used to implement a wide range of integration scenarios and workflows.

Key concepts and terminologies for apache camel:

Some of the key concepts and terminology are:-

- **Route** - A route in Apache Camel is a sequence of processing steps that a message follows from a source to a destination. A route typically consists of one or more endpoints, processors, and transformers.
- **Endpoint** - An endpoint in Apache Camel represents a source or destination of messages. It can be a file, a database, a messaging system, or any other system that can send or receive messages.
- **Processor** - A processor in Apache Camel is a component that can transform or manipulate messages as they pass through a route. Processors can be used to enrich, filter, aggregate, or transform messages, among other things.
- **Component** - A component in Apache Camel is a pre-built module that provides integration with a specific system or protocol, such as HTTP, JMS, or FTP. Components can be used to easily integrate with external systems without having to write custom code.
- **Exchange** - An exchange in Apache Camel represents a message and its associated metadata as it moves through a route. It contains information such as the message body, message headers, and properties.

- **Message** - A message in Apache Camel represents a unit of data that is passed through a route. It can be any type of data, such as a file, a text message, or a Java object.
- **RouteBuilder** - A RouteBuilder in Apache Camel is a Java class that is used to define routes and configure the CamelContext. It allows developers to define routes and configure components and processors in a fluent and easy-to-read manner.
- **CamelContext** - A CamelContext in Apache Camel represents the runtime environment for a set of routes. It manages the lifecycle of routes and provides services such as error handling and message routing.
- **Predicate** - A predicate in Apache Camel is a component that can be used to evaluate whether a message should be processed or not. Predicates can be used to filter messages based on their content or metadata.
- **Transformer** - A transformer in Apache Camel is a component that can be used to convert a message from one format to another. Transformers can be used to convert messages between XML, JSON, or other formats.

What are various components in Apache camel? Some of them that I have used?

Apache camel provides us with a number of components. These components make interacting create endpoints with which a system can interact with other external systems. For example using an ActiveMQ component we expose an ActiveMQ endpoint for interaction with external systems. There are more than 100 components provided by Apache Camel. Some of them are MAIL, JETTY, JMS, FILE, TIMMER, MARSHAL, UNMARSHAL etc. Apache camel also allows users to create custom components.

Mail Component:-In Apache Camel, the Mail component is used to send and receive emails from different mail servers using the Simple Mail Transfer Protocol (SMTP) and Internet Message Access Protocol (IMAP) protocols.

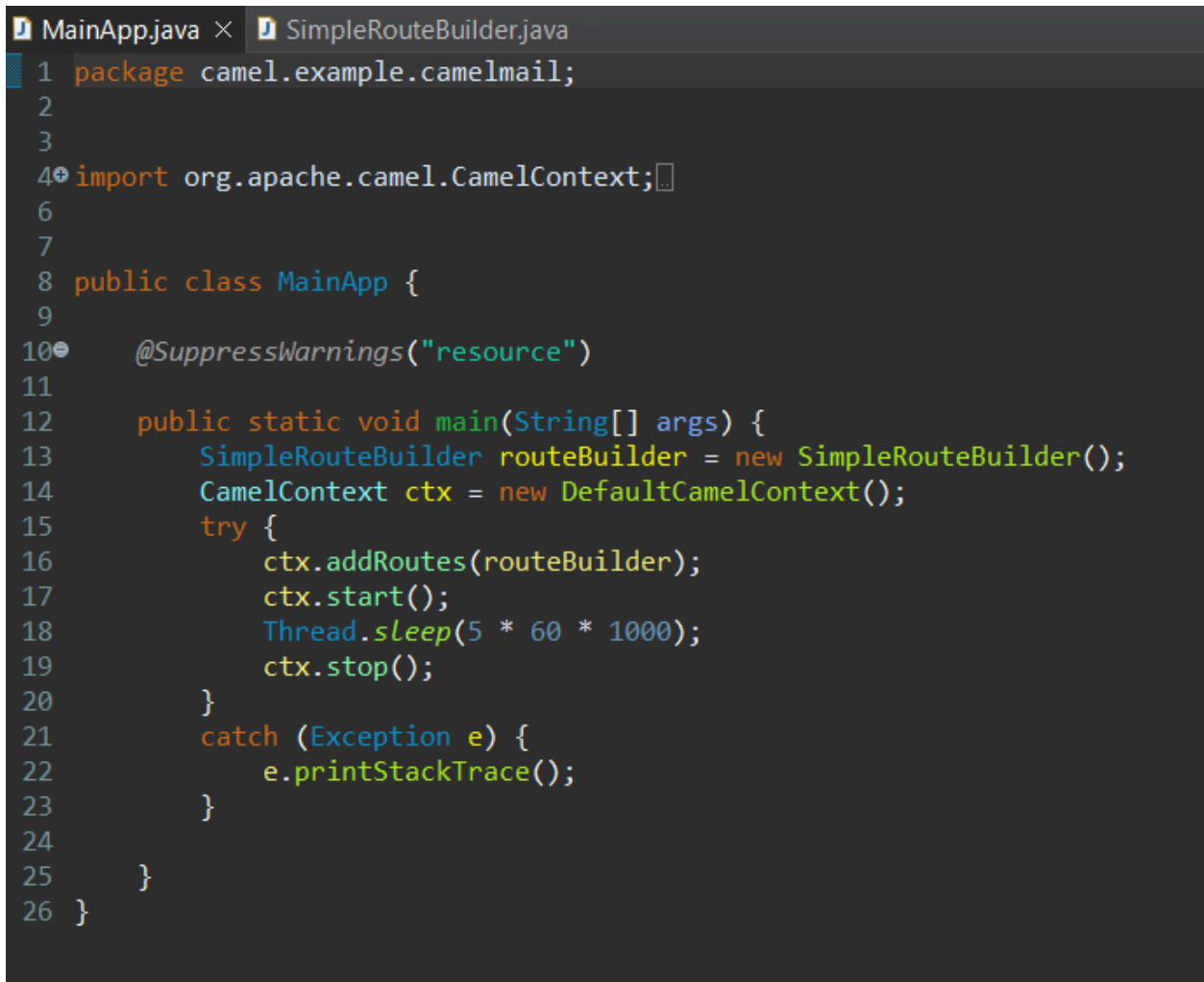
To use the Mail component, you first need to define a mail endpoint which specifies the mail server, username, password, and other settings such as the protocol to use, port number, and security options. Here's an example of a simple mail endpoint definition:

Dependency for mail component:-

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-mail</artifactId>
  <version>x.x.x</version>
  <!-- use the same version as your Camel core version -->
</dependency>
```

Here i am using **<version>3.20.1</version>**

In Springboot Application i have defined main class to run an application for example:-



```
1 package camel.example.camelmail;
2
3
4 import org.apache.camel.CamelContext;
5
6
7
8 public class MainApp {
9
10     @SuppressWarnings("resource")
11
12     public static void main(String[] args) {
13         SimpleRouteBuilder routeBuilder = new SimpleRouteBuilder();
14         CamelContext ctx = new DefaultCamelContext();
15         try {
16             ctx.addRoutes(routeBuilder);
17             ctx.start();
18             Thread.sleep(5 * 60 * 1000);
19             ctx.stop();
20         }
21         catch (Exception e) {
22             e.printStackTrace();
23         }
24
25     }
26 }
```

On the other hand i have created one class for mail component for that here is an example:-

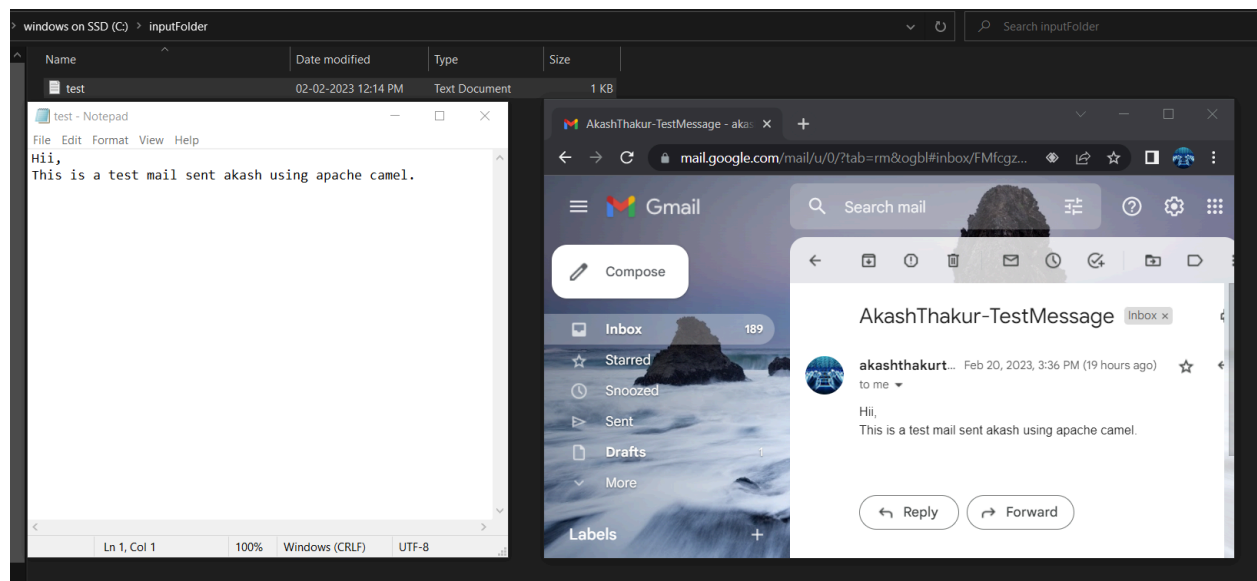
Example1)

```

1 package camel.example.camelmail;
2
3 import org.apache.camel.builder.RouteBuilder;
4
5 public class SimpleRouteBuilder extends RouteBuilder {
6
7     @Override
8     public void configure() throws Exception {
9
10         from("file:C:\\inputFolder?noop=true")
11             .doTry().setHeader("subject", simple("AkashThakur-TestMessage"))
12             .setHeader("to", simple("akashthakurthakur111@gmail.com,testouthworking@gmail.com"))
13             .to("smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvps1dlqkqzjmct");
14     }
15 }
16

```

In this example, the from route receives a message from the C:\input folder and sets the message headers for the recipient, and recipient mail address and also sender addresses. The message is then sent to the mail server specified in the endpoint URI, which includes the mail server hostname and port number, along with the username and password required for authentication. When i run this as a java application I got an email.



For Password :- Go to 1) Manage your google account 2) Security 3) Signing in to Google 3a) by two step verification you need to generate your mail password and

specify in password field.

Signing in to Google



Password	Last changed Oct 27, 2022	>
2-Step Verification	☒ On	>
App passwords	2 passwords	>

Specify this Line in application Property 

`spring.main.allow-circular-references=true`

Example 2)

```
from("direct:sendMail")
    .setHeader("To", constant("recipient@example.com"))
    .setHeader("From", constant("sender@example.com"))
    .setBody(constant("Hello, this is a test email"))
    .to("smtps://smtp.gmail.com:465?username=myusername&password=mypassword");
```

In this example, the from route receives a message and sets the message headers for the recipient and sender addresses, as well as the email body. The message is then sent to the mail server specified in the endpoint URI, which includes the mail server hostname and port number, along with the username and password required for authentication.

1. Jetty Component :-(Java Dsl)

In Apache Camel, the Jetty component is used to expose HTTP endpoints and create HTTP clients. The Jetty component provides a powerful, lightweight, and embedded HTTP server that can be used to build RESTful web services, serve static content, and proxy HTTP requests.

To use the Jetty component, you first need to configure a Jetty endpoint which specifies the HTTP server configuration, such as the hostname, port number, and SSL settings. Here's an example of a simple Jetty endpoint definition:

Dependency for jetty component:-

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-jetty</artifactId>
  <version>x.x.x</version>
  <!-- use the same version as your Camel core version -->
</dependency>
```

Simple Example for jetty:-

```
from("jetty:http://localhost:8080/myservice")
.to("log:receivedMessage");
```

Full Example:-

In Springboot Application i have defined main class to run an application for example:-

```
1 package camel.example.camelmail;
2
3
4 import org.apache.camel.CamelContext;
5
6
7
8 public class MainApp {
9
10     @SuppressWarnings("resource")
11
12     public static void main(String[] args) {
13         SimpleRouteBuilder routeBuilder = new SimpleRouteBuilder();
14         CamelContext ctx = new DefaultCamelContext();
15         try {
16             ctx.addRoutes(routeBuilder);
17             ctx.start();
18             Thread.sleep(5 * 60 * 1000);
19             ctx.stop();
20         }
21         catch (Exception e) {
22             e.printStackTrace();
23         }
24     }
25 }
26 }
```

On the other hand i have created one class for mail & jetty component for that here is an example:-

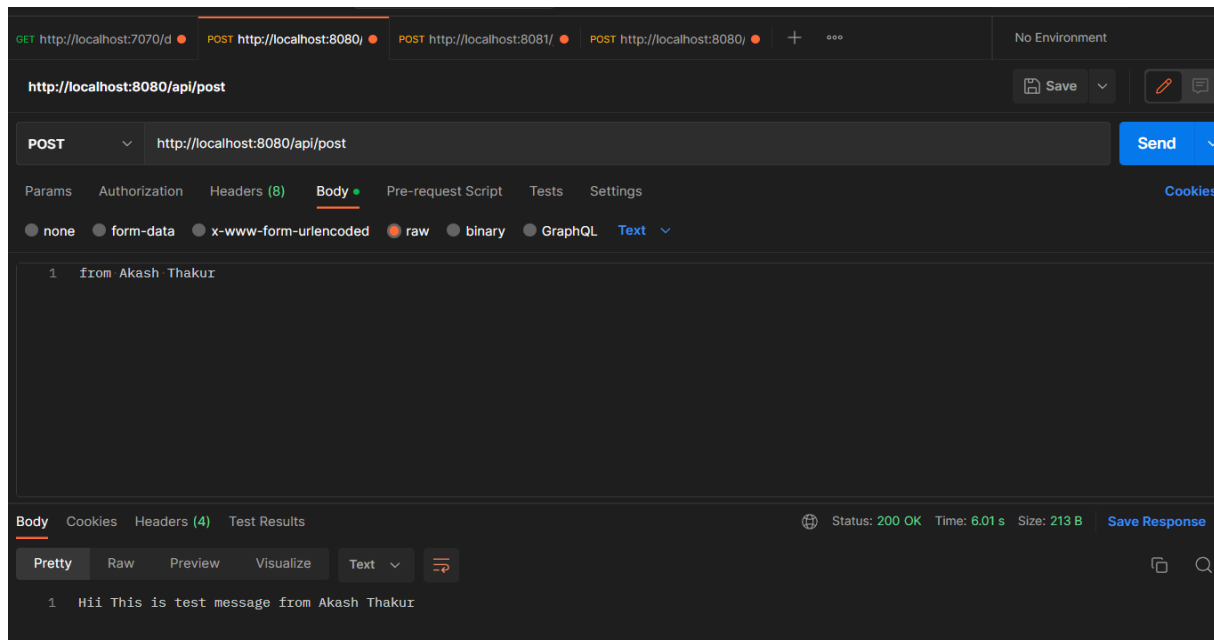
```
import org.apache.camel.builder.RouteBuilder;

public class SimpleRouteBuilder extends RouteBuilder {

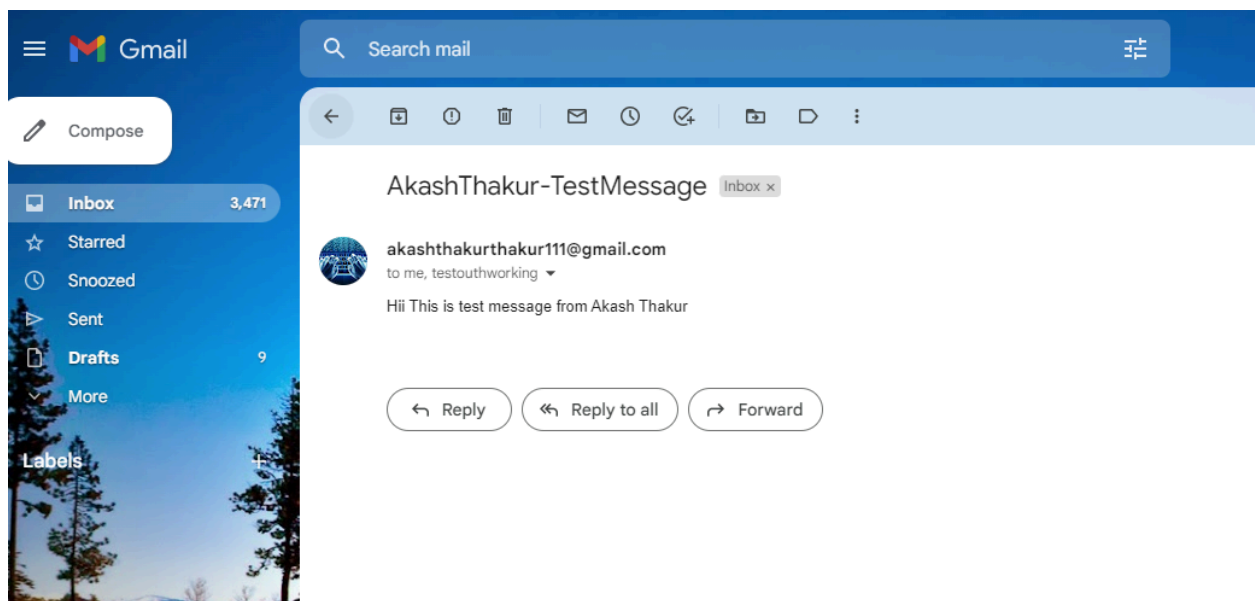
    @SuppressWarnings("deprecation")
    @Override
    public void configure() throws Exception {

        from("jetty:http://0.0.0.0:8080/api/post")
            .process(exchange -> {
                String body = exchange.getIn().getBody(String.class);
                // do some processing with the body
                exchange.getOut().setBody("Hi! This is test message " + body);
            })
            .doTry()
            .setHeader("subject", simple("AkashThakur-TestMessage"))
            .setHeader("to", simple("akashrthakur786@gmail.com,testouthworking@gmail.com"))
            .to("smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkqzjmnct");
        // .to("smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=bzdzxbfihdzrhavt&to=");
    }
}
```

In this example, the from route exposes an HTTP endpoint at <http://localhost:8080/api/post> using the Jetty server. Any HTTP requests to this endpoint will trigger the route, and the received message will be sent to the mail endpoint.



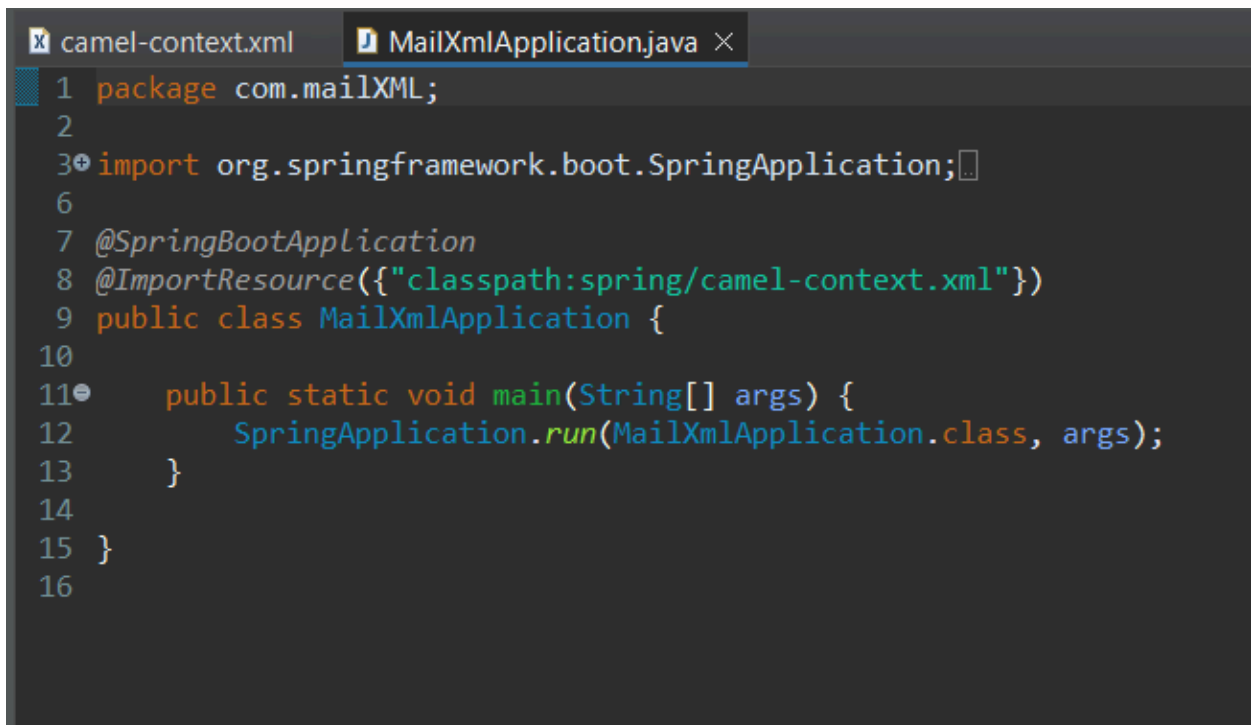
In this example I am hitting the API endpoint to send the body message.



Here in this mail you will see the message received from Api hit, the **Hii This is test message** is taken from The Body of the code And the **From Akash Thakur** text is taken from api body, both are sent to my mail id as **Hii This is text message from Akash Thakur**.

Same Example I am writing in Xml Dsl

2. Jetty Component (Xml Dsl):-



```
1 package com.mailXML;
2
3 import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 @ImportResource({"classpath:spring/camel-context.xml"})
8 public class MailXmlApplication {
9
10
11     public static void main(String[] args) {
12         SpringApplication.run(MailXmlApplication.class, args);
13     }
14
15 }
16
```

Main Class in Java to run Xml Dsl, here in line 8 i am importing the Xml file to call xml file while running this application.

The Below example is the Xml Dsl that we are importing in Main.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         http://www.springframework.org/schema/beans/spring-beans.xsd
8         http://camel.apache.org/schema/spring
9         http://camel.apache.org/schema/spring/camel-spring.xsd">
10
11   <camelContext id="myCamel" xmlns="http://camel.apache.org/schema/spring">
12
13     <route id="apiroute">
14       <from uri="jetty:http://localhost:8081/jetty"/>
15       <setBody>
16         <simple>Hello, email recipient! This message was received from Jetty.</simple>
17       </setBody>
18       <to uri="smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkqzjmnet&to=akashthakurthakur111@gmail.com&subject=Message from Jetty&from=akashthakur111@gmail.com&debugMode=true"/>
19     </route>
20   </camelContext>
21 </beans>
```

Code.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd
         http://camel.apache.org/schema/spring
         http://camel.apache.org/schema/spring/camel-spring.xsd">
```

```
<camelContext id="myCamel" xmlns="http://camel.apache.org/schema/spring">
```

```
<route id="apiroute">
  <from uri="jetty:http://localhost:8081/jetty"/>
  <setBody>
    <simple>Hello, email recipient! This message was received from Jetty.</simple>
  </setBody>
  <to
```

```
uri='smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkqzjmnet&to=akashthakurthakur111@gmail.com&subject=Message from Jetty&from=akashthakurthakur111@gmail.com&debugMode=true'/>
  </route>
```

```
</camelContext>
```

```
</beans>
```

This Is my pom.Xml

```
mailXML/pom.xml ×
1 |<?xml version="1.0" encoding="UTF-8"?>
2 |<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4 |   <modelVersion>4.0.0</modelVersion>
5 |   <parent>
6 |     <groupId>org.springframework.boot</groupId>
7 |     <artifactId>spring-boot-starter-parent</artifactId>
8 |     <version>2.7.8</version>
9 |     <relativePath/> <!-- lookup parent from repository -->
10 |   </parent>
11 |   <groupId>com.mailXML</groupId>
12 |   <artifactId>mailXML</artifactId>
13 |   <version>0.0.1-SNAPSHOT</version>
14 |   <name>mailXML</name>
15 |   <description>mailXML project for Spring Boot</description>
16 |   <properties>
17 |     <java.version>19</java.version>
18 |   </properties>
19 |   <dependencies>
20 |     <dependency>
21 |       <groupId>org.springframework.boot</groupId>
22 |       <artifactId>spring-boot-starter-web</artifactId>
23 |     </dependency>
24 |     <dependency>
25 |       <groupId>org.apache.camel.springboot</groupId>
26 |       <artifactId>camel-spring-boot-starter</artifactId>
27 |       <version>3.20.1</version>
28 |     </dependency>
29 |     <dependency>
30 |       <groupId>org.apache.camel</groupId>
31 |       <artifactId>camel-jetty</artifactId>
32 |       <version>3.20.1</version>
33 |     </dependency>
34 |     <dependency>
35 |       <groupId>org.apache.camel</groupId>
36 |       <artifactId>camel-mail</artifactId>
37 |       <version>3.20.1</version>
38 |     </dependency>
39 |
40 |     <dependency>
41 |       <groupId>org.springframework.boot</groupId>
42 |       <artifactId>spring-boot-starter-test</artifactId>
43 |       <scope>test</scope>
```

```
43         <scope>test</scope>
44       </dependency>
45     </dependencies>
46
47     <build>
48       <plugins>
49         <plugin>
50           <groupId>org.springframework.boot</groupId>
51           <artifactId>spring-boot-maven-plugin</artifactId>
52         </plugin>
53       </plugins>
54     </build>
55
56 </project>
57
```

1. Jms Component:-(Java Dsl)

JMS (Java Message Service) is a Java-based API (Application Programming Interface) that provides a common way for applications to create, send, receive, and read messages asynchronously between components in a distributed computing environment. A JMS component is a software component that implements the JMS API and enables communication between different applications or services using the messaging paradigm.

JMS components can be implemented using various messaging middleware technologies such as Apache ActiveMQ, IBM MQ, and RabbitMQ. JMS components can also be integrated with various enterprise technologies such as EJB (Enterprise Java Beans), JPA (Java Persistence API), and Spring Framework. So I am using ActiveMQ middleware.

Dependency For JMSComponent:-

```
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-jms</artifactId>
  <version>x.x.x</version>
  <!-- use the same version as your Camel core version -->
</dependency>
```

Here's an example of how to use JMS to send and receive messages between two Java applications using ActiveMQ as the messaging middleware:

1. First, you need to download and install ActiveMQ on your machine.
2. Create a Java project and add the ActiveMQ client library to the classpath.
3. Create a JMS connection factory object that connects to the ActiveMQ broker:
4. Create a message producer and send a message to the queue:
5. Create a message consumer and receive the message from the queue:
6. Close the JMS objects and the connection:

Here is a full example using the Jms component along with Mail & Jetty components.

This is my main class.

```
*CamelJettyJmsMailExample.java x
84 import org.apache.activemq.ActiveMQConnectionFactory;
89
90 public class CamelJettyJmsMailExample {
91
92     @SuppressWarnings("resource")
93     public static void main(String[] args) throws Exception {
94         CamelContext context = new DefaultCamelContext();
95
96         // Define the connection factory
97         ActiveMQConnectionFactory connectionFactory = new ActiveMQConnectionFactory("tcp://localhost:61616");
98         // Add the connection factory to the Camel context
99         context.addComponent("jms", JmsComponent.jmsComponentAutoAcknowledge(connectionFactory));
100
101         context.addRoutes(new RouteBuilder() {
102             public void configure() {
103                 from("jetty:http://localhost:8080/sendmail")
104                     .routeId("jettyroute")
105                     .to("jms:queue:testQueue")
106                     .log("The message is reached to the destion by jetty");
107             }
108         });
109
110         context.addRoutes(new RouteBuilder() {
111             public void configure() {
112                 from("jms:queue:testQueue")
113                     .routeId("mailroute")
114                     .to("smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkqzjmnct&subject=Testmessage");
115             }
116         });
117
118         context.start();
119         Thread.sleep(5 * 60 * 1000);
120         context.stop();
121     }
122 }
```

The screenshot shows a REST client interface with the following details:

- URL:** http://localhost:8080/sendmail
- Method:** POST
- Body:** hii postman akash
- Status:** 200 OK
- Time:** 6.37 s
- Size:** 699 B
- Response:** hii postman akash




[Home](#) | [Queues](#) | [Topics](#) | [Subscribers](#) | [Connections](#) | [Network](#) | [Scheduled](#) | [Send](#)

[Support](#) | [Logout](#)

Queue Name
Queue Name Filter

Queues:

Name	Number Of Pending Messages	Number Of Consumers	Messages Enqueued	Messages Dequeued	Views	Operations
testQueue	0	0	1	1	Browse Active Consumers Active Producers	Send To Purge Delete Pause

Queue Views

- Graph
- XML

Topic Views

- XML

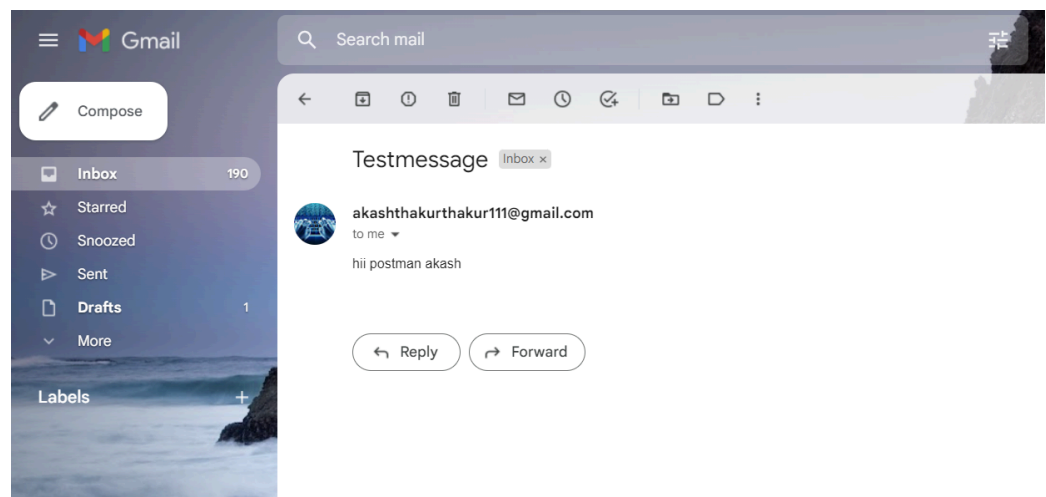
Subscribers Views

- XML

Useful Links

- Documentation
- FAQ
- Downloads
- Forums

Copyright 2005-2022 The Apache Software Foundation.



In this example I am specifying the ActiveMQ connectionfactory. for that I need to add the ActiveMQ client library to the classpath.

Create a JMS connection factory object that connects to the ActiveMQ broker:
With the port that connects to Activemq.

And I have Created two routers for One for jettyRoute that produce from api hit and consume from Activemq queue the queue name is testQueue as above, and other for mailRoute that consume from the activemq queue message received from api body and produce to smtp destination that we have specified.

This is Pom.Xml file

```
*CamelJettyJmsMailExample.java  camel-jms/pom.xml ×
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>2.7.8</version>
9         <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11    <groupId>camel-jms</groupId>
12    <artifactId>camel-jms</artifactId>
13    <version>0.0.1-SNAPSHOT</version>
14    <name>camel-jms</name>
15    <description>Jms-mail-jetty project for Spring Boot</description>
16    <properties>
17        <java.version>19</java.version>
18    </properties>
19    <dependencies>
20        <dependency>
21            <groupId>org.springframework.boot</groupId>
22            <artifactId>spring-boot-starter-web</artifactId>
23        </dependency>
24        <dependency>
25            <groupId>org.apache.camel.springboot</groupId>
26            <artifactId>camel-spring-boot-starter</artifactId>
27            <version>3.20.1</version>
28        </dependency>
29        <dependency>
30            <groupId>org.apache.camel</groupId>
31            <artifactId>camel-jetty</artifactId>
32            <version>3.20.1</version>
33        </dependency>
34        <dependency>
35            <groupId>org.apache.camel</groupId>
36            <artifactId>camel-mail</artifactId>
37            <version>3.20.1</version>
38        </dependency>
39        <dependency>
40            <groupId>org.apache.camel</groupId>
41            <artifactId>camel-jms</artifactId>
42            <version>3.20.1</version>
43        </dependency>
```

```

43     </dependency>
44     <dependency>
45         <groupId>org.apache.camel</groupId>
46         <artifactId>camel-activemq</artifactId>
47         <version>3.20.1</version>
48         <!-- use the same version as your Camel core version -->
49     </dependency>
50     <dependency>
51         <groupId>org.apache.activemq</groupId>
52         <artifactId>activemq-camel</artifactId>
53         <version>5.16.5</version><!--$NO-MVN-MAN-VER$-->
54     </dependency>
55     <dependency>
56         <groupId>org.apache.activemq</groupId>
57         <artifactId>activemq-broker</artifactId>
58         <version>5.16.5</version><!--$NO-MVN-MAN-VER$-->
59     </dependency>
60
61     <dependency>
62         <groupId>org.springframework.boot</groupId>
63         <artifactId>spring-boot-starter-test</artifactId>
64         <scope>test</scope>
65     </dependency>
66 </dependencies>
67
68 <build>
69     <plugins>
70         <plugin>
71             <groupId>org.springframework.boot</groupId>
72             <artifactId>spring-boot-maven-plugin</artifactId>
73         </plugin>
74     </plugins>
75 </build>
76
77 </project>
78

```

2. Jms Component:-(Xml Dsl)

This is main class in Springboot

```

*CamelJmsXmlApplication.java ×  *camel-context.xml
1 package com.jmsXML;
2
3 import org.springframework.boot.SpringApplication;
4
5
6
7 @SpringBootApplication
8 @ImportResource({"classpath:spring/camel-context.xml"})
9 public class CamelJmsXmlApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(CamelJmsXmlApplication.class, args);
13     }
14
15 }
16

```

This is an xml file for jms,jetty,mail component.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         http://www.springframework.org/schema/beans/spring-beans.xsd
8         http://camel.apache.org/schema/spring
9         http://camel.apache.org/schema/spring/camel-spring.xsd">
10
11   <bean id="jmsConnectionFactory" class="org.apache.activemq.ActiveMQConnectionFactory">
12     <property name="brokerURL" value="tcp://localhost:61616" />
13   </bean>
14
15   <bean id="jmsConfig" class="org.apache.camel.component.jms.JmsConfiguration">
16     <property name="connectionFactory" ref="jmsConnectionFactory" />
17     <property name="concurrentConsumers" value="10" />
18   </bean>
19
20
21   <camelContext id="myCamel" xmlns="http://camel.apache.org/schema/spring">
22     <route id="apiroute">
23       <from uri="jetty:http://localhost:8081/sendmail"/>
24       <to uri='jms:queue:testQueue' />
25     </route>
26
27     <route id="mailroute">
28       <from uri="jms:queue:testQueue"/>
29       <to uri='smtps://smtp.gmail.com:465?username=akashthakur111@gmail.com&password=jvpsldlqkqzjmnct&to=akashthakurthakur' />
30     </route>
31   </camelContext>
32
```

This xml file we are able to get the same output as java dsl.

Handling of DLQ (Dead-letter queue)

A dead-letter queue (DLQ), sometimes referred to as an undelivered-message queue, is a holding queue for messages that cannot be delivered to their destination queues, for example because the queue does not exist, or because it is full.

To handle DLQ in above xml.dsl i am not providing any destination or not specified any mailroute for it,so it will fail to deliver message to destination after that, once the message entered into DLQ then the consumer consumes from dlq and produces to the destination.as below example.

I had some changes in the xml file as the example below you will be able to see.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         http://www.springframework.org/schema/beans/spring-beans.xsd
8         http://camel.apache.org/schema/spring
9         http://camel.apache.org/schema/spring/camel-spring.xsd">
10
11 <bean id="jmsConnectionFactory" class="org.apache.activemq.ActiveMQConnectionFactory">
12   <property name="brokerURL" value="tcp://localhost:61616" />
13 </bean>
14
15 <bean id="jmsConfig" class="org.apache.camel.component.jms.JmsConfiguration">
16   <property name="connectionFactory" ref="jmsConnectionFactory" />
17   <property name="concurrentConsumers" value="10" />
18 </bean>
19
20 <camelContext id="myCamel" xmlns="http://camel.apache.org/schema/spring">
21   <route id="apiroute">
22     <from uri="jetty:http://localhost:8081/sendmail"/>
23     <to uri="jms:queue:testQueue"/>
24   </route>
25
26   <route id="dlqroute">
27     <from uri="jms:queue:ActiveMQ.DLQ"/>
28     <to uri="smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkqzjmnct&to=akashthakurthakur111@gmail.com"/>
29   </route>
30 </camelContext>
31
32 </beans>

```

When i run this application and hit the api that it will send the body message to Activemq queue from there it will not found any destination so it will show failed to send message in postman but that consumed message is stored in Dlq, so we will consume from dlq and produce to the destination as our email id.as below example

In postman you will see it has failed to post, but Activemq consumed this message & passed to Dlq.

The screenshot shows a Postman interface for a POST request to `http://localhost:8081/sendmail`. The request body is `hii postman akash`. The response status is `500 Server Error` with a time of `21.37 s` and size of `115 KB`. The response body, shown in 'Pretty' format, is a Java exception stack trace:

```

1 org.apache.camel.ExchangeTimeoutException: The OUT message was not received within: 20000 millis due reply message with correlationID:
  Camel-31B42FAA8B8510D-0000000000000001 not received on destination: temp-queue://ID:DESKTOP-R11C2BS-65489-1676975484252-1:2:1. Exchange
  [31B42FAA8B8510D-0000000000000000]
2   at org.apache.camel.component.jms.reply.ReplyManagerSupport.processReply(ReplyManagerSupport.java:187)
3   at org.apache.camel.component.jms.reply.TemporaryQueueReplyHandler.onTimeout(TemporaryQueueReplyHandler.java:62)
4   at org.apache.camel.component.jms.reply.CorrelationTimeoutMap.lambda$new$0(CorrelationTimeoutMap.java:41)
5   at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:577)
6   at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:317)
7   at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1144)
8   at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:642)
9   at java.base/java.lang.Thread.run(Thread.java:1589)

```

In the console also you will be able to see how it failed to send and how it is reprocessing from dlq to send the same message body to email.

```
2023-02-21 16:00:04.361 INFO 2508 --- [main] o.a.c.impl.engine.AbstractCamelContext : Routes startup (started:2)
2023-02-21 16:00:04.506 INFO 2508 --- [main] o.a.c.impl.engine.AbstractCamelContext : Started apiroute (jetty:http://localhost:8081/sendmail)
2023-02-21 16:00:04.508 INFO 2508 --- [main] o.a.c.impl.engine.AbstractCamelContext : Started dlqroute (jms://queue:ActiveMQ.DLQ)
2023-02-21 16:00:04.509 INFO 2508 --- [main] o.a.c.impl.engine.AbstractCamelContext : Apache Camel 3.20.1 (myCamel) started in 2s18ms (build:334ms init:709ms start:975ms)
2023-02-21 16:00:04.602 INFO 2508 --- [main] com.jmsXML.CamelJmsXMLApplication : Started CamelJmsXMLApplication in 17.495 seconds (JVM running for 19.512)
2023-02-21 16:00:52.243 WARN 2508 --- [eout[testQueue]] o.a.c.c.j.r.TemporaryQueueReplyManager : Timeout occurred after 20000 millis waiting for reply message with correlationID [Camel-31B42FAA888510D-00000000000000000000000000000000]
2023-02-21 16:00:52.278 ERROR 2508 --- [eout[testQueue]] o.a.c.p.e.DefaultErrorHandler : Failed delivery for (MessageId: 31B42FAA888510D-00000000000000000000000000000000 on ExchangeId: 31B42FAA888510D-00000000000000000000000000000000)

Message History (source location and message history is disabled)
-----
Source                                     ID                                     Processor                                     Elapsed (ms)
-----
...                                         apiroute/apiroute                     from[jetty:http://localhost:8081/sendmail] 345776446
                                         apiroute/tol                           jms:queue:testQueue                        0

Stacktrace
-----
org.apache.camel.ExchangeTimeoutException: The OUT message was not received within: 20000 millis due reply message with correlationID: Camel-31B42FAA888510D-00000000000000000000000000000000 not received
    at org.apache.camel.component.jms.reply.ReplyManagerSupport.processReply(ReplyManagerSupport.java:187) ~[camel-jms-3.20.1.jar:3.20.1]
    at org.apache.camel.component.jms.reply.TemporaryQueueReplyHandler.onTimeout(TemporaryQueueReplyHandler.java:62) ~[camel-jms-3.20.1.jar:3.20.1]
    at org.apache.camel.component.jms.reply.CorrelationTimeoutMap.lambda$new$0(CorrelationTimeoutMap.java:41) ~[camel-jms-3.20.1.jar:3.20.1]
    at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:577) ~[na:na]
    at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:317) ~[na:na]
    at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1144) ~[na:na]
    at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:642) ~[na:na]
    at java.base/java.lang.Thread.run(Thread.java:1582) ~[na:na]

DEBUG: getProvider() returning javax.mail.Provider[TRANSPORT,smtps,com.sun.mail.smtp.SMTPSSLTransport,Oracle]
DEBUG SMTP: useEhlo true, useAuth true
DEBUG SMTP: trying to connect to host "smtp.gmail.com", port 465, isSSL true
220 smtp.gmail.com ESMTP i76-20020a636d4f000000b004fbd91d9716sm2553752pgc.15 - gsmt
DEBUG SMTP: connected to host "smtp.gmail.com", port: 465
EHLO DESKTOP-R11C2B5
250-smtp.gmail.com at your service, [119.82.111.14]
250-SIZE 35882577
250-8BITIME
250-AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH
250-ENHANCEDSTATUSCODES
250-PIPELINING
250-CHUNKING
250-SMTPUTF8
DEBUG SMTP: Found extension "SIZE", arg "35882577"
DEBUG SMTP: Found extension "8BITIME", arg ""
```

In Activemq See how it is delivered from DLQ.



Home | Queues | Topics | Subscribers | Connections | Network | Scheduled | Send

Support | Logout

Queue Name Create Queue Name Filter Filter

Queues:

Name	Number Of Pending Messages	Number Of Consumers	Messages Enqueued	Messages Dequeued	Views	Operations
ActiveMQ.DLQ	0	0	1	1	Browse Active Consumers Active Producers	Send To Purge Delete Pause
testQueue	0	0	1	1	Browse Active Consumers Active Producers	Send To Purge Delete Pause

Queue Views

[Graph](#)
[XML](#)

Topic Views

[XML](#)

Subscribers Views

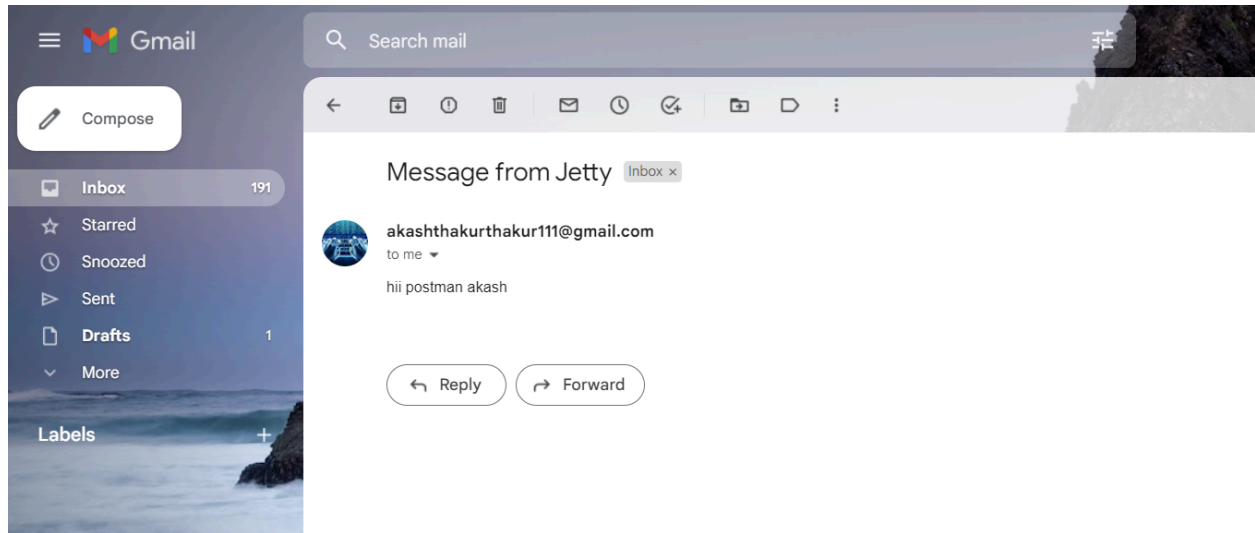
[XML](#)

Useful Links

[Documentation](#)
[FAQ](#)
[Downloads](#)
[Forums](#)

Copyright 2005-2022 The Apache Software Foundation.

Here I got the mail from Activemq Dlq.



Using Marshal and Unmarshal(Data transformation):-

Camel has support for message transformation using several techniques. One such technique is [Data Formats](#), where marshal and unmarshal comes from.

So in other words the [Marshal](#) and [Unmarshal](#) EIPs are used with [Data Formats](#).

- *Marshal* - Transforms the message body (such as Java object) into a binary or textual format, ready to be wired over the network.
- *Unmarshal* - Transforms data in some binary or textual format (such as received over the network) into a Java object; or some other representation according to the data format being used.

Error Handling in Apache camel using doTry() and doCatch()

- Error handling for RuntimeException
- Error handling for MessageException
- Error handling for other Exception

Error handling and marshal and unmarshal in my code using some of the components like mail,jetty,Jms,Activemq,Marshal,Unmarshal.

Java Dsl Example

```
import org.apache.activemq.ActiveMQConnectionFactory;
import org.apache.camel.CamelContext;
import org.apache.camel.LoggingLevel;
import org.apache.camel.impl.DefaultCamelContext;
import org.apache.camel.model.dataformat.JsonLibrary;
import org.springframework.core.annotation.Order;

import com.fasterxml.jackson.databind.SerializationFeature;

import org.apache.camel.builder.RouteBuilder;
import org.apache.camel.component.jackson.JsonDataFormat;
import org.apache.camel.component.jms.JmsComponent;

public class CamelJettyJmsMailExample {

    @SuppressWarnings("resource")
    public static void main(String[] args) throws Exception {
        CamelContext context = new DefaultCamelContext();

        // Define the connection factory
        ActiveMQConnectionFactory connectionFactory = new
ActiveMQConnectionFactory("tcp://localhost:61616");
        // Add the connection factory to the Camel context
        context.addComponent("jms", JmsComponent.jmsComponentAutoAcknowledge(connectionFactory));

        JsonDataFormat jsonDataFormat = new JsonDataFormat();
        jsonDataFormat.disableFeature(SerializationFeature.FAIL_ON_EMPTY_BEANS);

        context.addRoutes(new RouteBuilder() {

            public void configure() {
                // JsonDataFormat jsonFormat = new JsonDataFormat();
                from("jetty:http://localhost:8080/sendmail")
                    .routeId("jettyroute")
                    .doTry()
                    .unmarshal().json()
                    .to("jms:queue:testQueue")
                    .doCatch(RuntimeException.class)
                    .log("Runtime exception occurred: ${exception.message}")
            }
        });
    }
}
```

```

        .endDoTry()
        .doCatch(Exception.class)
        .log("An error occurred: ${exception.message}")
        .endDoTry();
    };
});

context.addRoutes(new RouteBuilder() {
    public void configure() {
        from("jms:queue:testQueue")
        .routeId("mailroute")
        .doTry()
        .marshal().json()
        .to("log:mail?showBody=true")

        .to("smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkqzjmn
ct&subject=Testmessage")
        .log("Email sent successfully")
        .doCatch(MessagingException.class)
        .log(LoggingLevel.ERROR, "Error sending email: ${exception.message}")
        .endDoTry();
    }
});

context.start();
Thread.sleep(5 * 60 * 1000);
context.stop();
}
}

```

Xml Dsl Example

Main class

```

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ImportResource;

@SpringBootApplication
@ImportResource({"classpath:spring/camel-context.xml"})
public class CamelJmsXmlApplication {

    public static void main(String[] args) {
        SpringApplication.run(CamelJmsXmlApplication.class, args);
    }
}

```

}

Camel-context.xml file

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://camel.apache.org/schema/spring
           http://camel.apache.org/schema/spring/camel-spring.xsd">

    <bean id="jmsConnectionFactory" class="org.apache.activemq.ActiveMQConnectionFactory">
        <property name="brokerURL" value="tcp://localhost:61616" />
    </bean>

    <bean id="jmsConfig" class="org.apache.camel.component.jms.JmsConfiguration">
        <property name="connectionFactory" ref="jmsConnectionFactory" />
        <property name="concurrentConsumers" value="10" />
    </bean>

    <camelContext id="myCamel" xmlns="http://camel.apache.org/schema/spring">

        <dataFormats>
            <json id="jsonOrder" library="Jackson"/>
        </dataFormats>

        <route id="apiroute">
            <from uri="jetty:http://localhost:8081/sendmail"/>
            <doTry>

                <unmarshal>
                    <json library="Jackson" disableFeatures="FAIL_ON_EMPTY_BEANS"/>
                </unmarshal>
                <to uri='jms:queue:testQueue'/>
                <doCatch>
                    <exception>java.lang.Exception</exception>
                    <log message="Error occurred in application: ${exception.message}"/>
                </doCatch>
                <doCatch>
                    <exception>java.lang.RuntimeException</exception>
                    <log message="Error occurred at runtime: ${exception.message}"/>
                </doCatch>

            </doTry>
        </route>
    </camelContext>
</beans>
```

```

</doTry>
<!-- <to uri='jms:queue:testQueue'/> -->
    </route>

    <route id="mailroute">
        <from uri="jms:queue:testQueue"/>
        <doTry>
            <marshal>
                <json library="Jackson"/>
            </marshal>
            <to
uri='smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkq
zjmnct&to=akashthakurthakur111@gmail.com&subject=Message from
Jetty&from=akashthakurthakur111@gmail.com&debugMode=true'/>
                <doCatch>
                    <exception>javax.mail.MessagingException</exception>
                    <log message="Error occurred while sending email: ${exception.message}"/>
                </doCatch>
            </doTry>
            <!-- <to
uri='smtps://smtp.gmail.com:465?username=akashthakurthakur111@gmail.com&password=jvpsldlqkq
zjmnct&to=akashthakurthakur111@gmail.com&subject=Message from
Jetty&from=akashthakurthakur111@gmail.com&debugMode=true'/> -->
                </route>
            </camelContext>

</beans>

```

Required Libraries

```

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.apache.camel.springboot</groupId>
        <artifactId>camel-spring-boot-starter</artifactId>
        <version>3.20.1</version>
    </dependency>

    <dependency>
        <groupId>org.apache.camel</groupId>
        <artifactId>camel-jetty</artifactId>
        <version>3.20.1</version>
    </dependency>

```

```

<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-mail</artifactId>
  <version>3.20.1</version>
</dependency>
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-jms</artifactId>
  <version>3.20.1</version>
</dependency>
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-activemq</artifactId>
  <version>3.20.1</version>
  <!-- use the same version as your Camel core version -->
</dependency>
<dependency>
  <groupId>org.apache.activemq</groupId>
  <artifactId>activemq-camel</artifactId>
  <version>5.16.5</version><!--$NO-MVN-MAN-VERS-->
</dependency>
<dependency>
  <groupId>org.apache.activemq</groupId>
  <artifactId>activemq-broker</artifactId>
  <version>5.16.5</version><!--$NO-MVN-MAN-VERS-->
</dependency>
<dependency>
  <groupId>org.apache.camel</groupId>
  <artifactId>camel-jackson</artifactId>
  <version>3.20.1</version>
</dependency>
<!-- https://mvnrepository.com/artifact/javax.xml.bind/jaxb-api -->

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

```

//